

I/O Multiplexing

I/O Multiplexing is a technique that allows one thread/process to monitor **many file descriptors (fd)** at the same time using a single system call.

This is a major breakthrough for scalable server design.

Instead of:

None

1 thread per connection

You can use:

None

1 thread handling many connections

1. Why It Is Needed

Problems with polling:

None

Repeatedly checking every socket wastes CPU

Problems with blocking I/O:

None

One waiting socket can block one thread

I/O multiplexing solves both better.

2. Core Idea

The OS monitors many sockets and tells the application:

None

These sockets are ready now

Ready means:

- Readable
- Writable
- New connection pending
- Error/hangup

3. How select() Works

A classic system call:

None

select()

You provide:

- Read fd set
- Write fd set
- Error fd set
- Timeout

OS blocks until at least one fd becomes ready.

Then returns:

None

Number of ready descriptors

4. Example Flow

None

Sockets: A, B, C, D

`select()` waits...

B readable

D writable

`select()` returns 2

Now application checks which sockets are ready.

5. Why This Is Better Than Polling

Instead of constantly checking:

None

A ready?

B ready?

C ready?

The thread sleeps efficiently until OS notifies readiness.

This reduces wasted CPU cycles.

6. Still Requires Readiness Scan

After `select()` returns, the program still loops through descriptors to find ready ones.

None

Need to inspect fd sets

So it is improved, but not perfect for huge scale.

7. Blocking vs Non-Blocking Clarification

The monitored sockets may be non-blocking, but:

None

```
select() itself can block waiting for events
```

That is efficient blocking, not wasteful blocking.

8. Why It Is Called Multiplexing

Because one thread multiplexes time across many connections.

None

```
Single worker handles multiple I/O channels
```

9. Limitations of select()

FD Count Limits

Traditional `select()` has maximum descriptor limits.

O(n) Scanning Cost

Must scan many descriptors every call.

Rebuild Sets Each Call

Need to reset fd sets repeatedly.

10. Better Modern Alternatives

For high-scale systems:

- `poll()`
- `epoll()` (Linux)
- `kqueue()` (BSD/macOS)
- IOCP (Windows)

Used by:

- Redis
- NGINX
- Node.js

11. Why It Matters in System Design

Enables:

- Thousands of concurrent sockets
- Lower thread count
- Lower memory usage
- Better throughput

12. Comparison

Model	CPU Efficiency	Thread Count	Scalability
Blocking Threads	Medium	High	Poor
Polling Non-blocking	Poor	Low	Medium
I/O Multiplexing	High	Low	Good
epoll/kqueue	Very High	Low	Excellent

Mental Model

None

Ask OS to watch many sockets,
wake me only when something is ready.

One-Line Summary

I/O multiplexing allows a single thread to efficiently manage many sockets by asking the operating system to wait for readiness events, greatly improving concurrency over thread-per-connection models.